Deep Learning Lecture: Gradient-Based Learning, Output Units and ReLU

Refai Arafat Rossi

October 2026

We will use one simple cybersecurity classification problem throughout the lesson: building a neural network that classifies network traffic as benign or malicious, and then extends the same task to distinguish benign traffic, DDoS attacks and port scans.
This single example will help your students understand why a neural network needs a cost function, why it uses different output units for different tasks, how hidden units extract features, and how ReLU supports gradient-based learning. These explanations follow the organization and core equations of Goodfellow's Chapter 6.

1 Start with the problem: How can a neural network recognize an attack?

Suppose a network administrator wants an AI model to detect suspicious traffic. We provide two numerical features for every network-flow sample.

Feature	**Meaning**	**Example value**
x_1	Normalized packet-rate feature	2
x_2	Normalized connection-failure feature	1

The model receives the input vector

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

For our first task, the target is:

$$y = \begin{cases} 0 & \text{Benign traffic} \\ 1 & \text{Malicious traffic} \end{cases}$$

Assume the known training label is $y = 1$. The model must learn from examples to predict the probability $P(y = 1 \mid \mathbf{x})$ rather than merely produce an arbitrary numerical score.
Our four main questions are:

1. How does the model produce a probability?
2. How do we measure the difference between its prediction and the correct answer?
3. How does the model change its weights to reduce that error?
4. Why does it need hidden units and activation functions such as ReLU?

1.1 A simple neural network diagram

- **Input layer:** $\mathbf{x} = [2, 1]^T$ (Packet-rate and connection-failure features)
- **Hidden layer (ReLU):** $\mathbf{h} = \text{ReLU}(W^T\mathbf{x} + \mathbf{b})$ (Transforms the input into learned features)
- **Output layer:** Sigmoid for binary classification, or Softmax for multiclass classification
- **Cost function:** $J(\theta) = -\log P(y \mid \mathbf{x}; \theta)$ (Measures the loss for the known training label)

The neural network makes a prediction in the forward pass. It then calculates the cost, computes gradients and adjusts its parameters during training. The essential idea in Section 6.2 is that a neural network is trained by defining a model, choosing a cost function and using an optimization procedure to reduce that cost.

2 Section 6.2 — Gradient-based learning

2.1 What does gradient-based learning mean?

A neural network contains trainable parameters, including weights and biases. Collect them into a parameter vector θ. The network's prediction depends on these parameters:

$$\hat{y} = f(\mathbf{x}; \theta)$$

The cost function tells us how poorly the prediction matches the target:

$$J(\theta) = L\big(f(\mathbf{x}; \theta), y\big)$$

The training objective is to find parameters that minimize the cost:

$$\boxed{\theta^* = \arg\min_{\theta} J(\theta)}$$

A gradient tells us how the cost changes as the parameters change:

$$\nabla_\theta J(\theta)$$

The basic gradient-descent update is

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta J}$$

where η is the learning rate.
Think of the cost as the height of a landscape. The gradient indicates the direction of the steepest increase, so subtracting the gradient moves the parameters locally in the direction of decreasing cost.

2.2 What happens during training?

1. **Step 1: Forward pass.** Process the input through the layers and calculate the prediction.
2. **Step 2: Calculate the cost.** Compare the predicted probability with the known label using a suitable loss function.
3. **Step 3: Calculate gradients.** Determine how the cost changes with the weights and biases.

4. **Step 4: Update the parameters.** Use gradient descent or an optimizer such as SGD to adjust the parameters, then repeat.

These steps repeat across many training examples. The detailed algorithm for efficiently calculating gradients throughout a network is backpropagation, covered separately in Goodfellow's Section 6.5.

2.3 The cost function in Goodfellow's framework

Goodfellow expresses the maximum-likelihood training objective as

$$\boxed{J(\theta) = -\mathbb{E}_{(\mathbf{x},y)\sim\hat{p}_{\text{data}}} \left[\log p_{\text{model}}(y \mid \mathbf{x};\theta)\right]}$$

Here:

- $\hat{p}_{\text{data}}$ represents the empirical distribution of the training data.
- $p_{\text{model}}(y \mid \mathbf{x};\theta)$ is the probability assigned by the model to the correct target.
- The negative logarithm penalizes the model when it assigns a low probability to the correct answer.

For a dataset containing N examples, the empirical cost is commonly written as

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\text{model}}(y_i \mid \mathbf{x}_i;\theta)$$

This is the negative log-likelihood, equivalent to the appropriate cross-entropy objective for the modeled output distribution.

3 Section 6.2.1 — Cost function: Cross-entropy and logistic regression

Now consider the binary traffic-classification problem again. The model estimates the probability that the traffic is malicious. The true class is $y = 1$. Suppose the model assigns a probability of 0.80 to the malicious class. How do we measure the quality of this prediction?

3.1 Cross-entropy loss for binary classification

For a binary target $y \in \{0, 1\}$, binary cross-entropy is

$$\boxed{L(y,p) = -\left[y \log p + (1-y)\log(1-p)\right]}$$

where $p = P(y = 1 \mid \mathbf{x})$. When the true label is $y = 1$, this expression simplifies to $L = -\log p$. For our example, $p = 0.80$:

$$L = -\ln(0.80) = \boxed{0.2231}$$

Now suppose the model is confidently wrong and predicts $p = 0.10$:

$$L = -\ln(0.10) = \boxed{2.3026}$$

The second model receives a much larger loss because it assigns a low probability to the correct class.

Teaching point: Cross-entropy does not simply ask whether the predicted class is right or wrong. It also considers the probability assigned to the correct answer. For easier classroom interpretation, these examples use natural logarithms and express loss in nats.

Predicted probability of malicious traffic	True label	Cross-entropy loss
0.95	1	0.0513
0.80	1	0.2231
0.50	1	0.6931
0.10	1	2.3026

3.2 What is logistic regression?

Logistic regression is a binary classification model. It first calculates a linear score and then converts it into a probability using the sigmoid function.
The linear score is

$$\boxed{z = \mathbf{w}^T\mathbf{x} + b}$$

The predicted probability is

$$\boxed{p = P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}}$$

The model can then classify the input according to a chosen decision threshold, often 0.5. Consider an example with $z = 1$:

$$p = \frac{1}{1 + e^{-1}} \approx 0.7311$$

The model predicts a 73.11% probability of malicious traffic. With true label $y = 1$, the loss becomes

$$L = -\ln(0.7311) \approx 0.3133$$

The combination of a sigmoid output and cross-entropy loss is a natural choice because it defines a probability distribution for the binary target and trains the model through maximum likelihood.

3.3 How is logistic regression related to a deep neural network?

In standard logistic regression, the linear score uses the original input features:

$$p = \sigma(\mathbf{w}^T\mathbf{x} + b)$$

In a neural network, the hidden layers first transform the input into a learned representation $\mathbf{h}$. The output layer then applies logistic regression to those learned features:

$$\boxed{p = \sigma(\mathbf{w}^T\mathbf{h} + b)}$$

The important difference is that the neural network learns a representation $\mathbf{h}$, while standard logistic regression directly uses $\mathbf{x}$. A neural network can therefore combine nonlinear hidden-layer transformations with a familiar logistic-regression output layer.

4 Section 6.2.2 — Output units

Goodfellow emphasizes that the choice of output units is closely connected to the cost function. The output layer must represent the type of target variable the model is learning to predict. For our network-security example, we will consider two different tasks:

- **Task A:** Is the traffic benign or malicious?
- **Task B:** Is the traffic benign, a DDoS attack or a port scan?

These tasks need different output distributions.

5 Section 6.2.2.2 — Sigmoid units for Bernoulli output distributions

5.1 When should we use sigmoid?

Use a sigmoid output when predicting a binary random variable, such as benign versus malicious, spam versus not spam, or disease detected versus not detected. A Bernoulli distribution is completely specified by one probability:

$$p = P(y = 1 \mid \mathbf{x})$$

The probability of the other outcome is $P(y = 0 \mid \mathbf{x}) = 1 - p$.
The sigmoid output unit is

$$\boxed{\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}}$$

where $z = \mathbf{w}^T\mathbf{h} + b$. The resulting Bernoulli distribution is

$$\boxed{P(y \mid \mathbf{x}) = \hat{y}^{\,y}(1 - \hat{y})^{1-y}}$$

Its negative log-likelihood gives the binary cross-entropy loss.

5.2 Understand the sigmoid graph

The sigmoid has an S-shaped curve. Its output lies strictly between 0 and 1 for finite inputs.

Logit z	**Sigmoid output**	**Interpretation**
-2	0.1192	Low probability of an attack
0	0.5000	Equal predicted probabilities
1	0.7311	Higher probability of an attack
2	0.8808	High probability of an attack

The sigmoid output is a probability estimate, not a guarantee that the prediction is correct. A threshold is needed to convert the probability into a class label. A threshold of 0.5 is common, but operational security systems may use a different threshold depending on the relative costs of false positives and false negatives.

5.3 Calculate one complete prediction

Return to the teaching example. Suppose the hidden layer produces features

$$\mathbf{h} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

For an illustrative set of output weights, let $z = 0.7h_1 + 0.4h_2 - 0.8$. Substituting the values:

$$z = 0.7(2) + 0.4(1) - 0.8 = 1$$

Therefore, $p = \sigma(1) = 0.7311$. For a malicious sample with $y = 1$:

$$L = -\ln(0.7311) = 0.3133$$

The model is leaning toward the correct answer, but the loss indicates that it has not assigned probability 1 to the target.

5.4 How does this prediction improve through gradient-based learning?

For sigmoid output combined with binary cross-entropy, the derivative of the loss with respect to the output logit simplifies to

$$\boxed{\frac{\partial L}{\partial z} = p - y}$$

In our example,

$$\frac{\partial L}{\partial z} = 0.7311 - 1 = -0.2689$$

The negative gradient indicates that increasing z locally decreases the loss.
To illustrate one update, let the learning rate be 0.1 and update only the output weights and bias. Their gradients are

$$\frac{\partial L}{\partial w_j} = (p - y)h_j, \qquad \frac{\partial L}{\partial b} = p - y$$

Parameter	Before update	After one update
w_1	0.7000	0.7538
w_2	0.4000	0.4269
b	-0.8000	-0.7731

The new logit is approximately 1.1614, producing $p_{\text{new}} \approx 0.7615$ and $L_{\text{new}} \approx -\ln(0.7615) = 0.2725$. The loss has decreased from approximately 0.3133 to 0.2725. This is one illustrative update; a complete training procedure also propagates gradients through the hidden layer.

6 Section 6.2.2.3 — Softmax units for multinoulli output distributions

Now change the classification task. Instead of two categories, we want the model to distinguish: 1. Benign traffic 2. DDoS attack 3. Port scan
These are mutually exclusive classes in this example: each input has one correct class. A single sigmoid output is no longer the natural way to represent the complete distribution over these three classes. Instead, we use a softmax output layer.

6.1 The softmax equation

Suppose the network produces K logits: $\mathbf{z} = [z_1, z_2, \ldots, z_K]$. Softmax converts these scores into probabilities:

$$\boxed{p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$$

The softmax probabilities satisfy two conditions: $0 < p_k < 1$ and

$$\boxed{\sum_{k=1}^{K} p_k = 1}$$

The scores before softmax are called logits. They are not probabilities; they can be any real numbers.

6.2 Work through a three-class example

Suppose the network produces these logits for a single traffic sample: Benign: 0, DDoS: 2, Port scan: 1. The exponentials are approximately $e^0 = 1$, $e^2 = 7.389$, $e^1 = 2.718$. Their sum is approximately 11.107. The probabilities are therefore:

$$P(\text{Benign} \mid \mathbf{x}) = \frac{1}{11.107} \approx 0.090$$

$$P(\text{DDoS} \mid \mathbf{x}) = \frac{7.389}{11.107} \approx 0.665$$

$$P(\text{Port scan} \mid \mathbf{x}) = \frac{2.718}{11.107} \approx 0.245$$

The largest probability is assigned to DDoS. The predicted class is therefore DDoS.

6.3 Softmax and cross-entropy

Suppose the true class is DDoS. Its one-hot target vector is $\mathbf{y} = [0, 1, 0]$. For a single-label, multiclass classification problem, cross-entropy is

$$\boxed{L = -\sum_{k=1}^{K} y_k \log p_k}$$

Since only the DDoS target entry equals 1,

$$L = -\ln(0.665) \approx \boxed{0.408}$$

The loss decreases when the model assigns a higher probability to the correct class.
For softmax combined with cross-entropy, the corresponding logit gradient is also simple:

$$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$$

6.4 Sigmoid versus softmax

Property	**Sigmoid output**	**Softmax output**
Typical application	Binary classification	Single-label multiclass classification
Output structure	One probability for positive class	One probability per class
Probability relationship	Negative class has probability $1 - p$	All class probabilities sum to 1
Distribution	Bernoulli	Multinoulli / categorical
Loss	Binary cross-entropy	Multiclass cross-entropy
Example	Benign versus attack	Benign, DDoS or port scan

Important distinction: If several labels can be true simultaneously, such as one traffic sample having multiple independent attributes, separate sigmoid outputs may be more appropriate than softmax. Softmax is intended for a probability distribution over mutually exclusive alternatives.

7 Section 6.3 — Hidden units

So far we have concentrated on the output layer. But where do the hidden features $\mathbf{h}$ come from? A hidden layer transforms the original input using weights, biases and an activation function. In Goodfellow's notation, a typical layer can be written as

$$\boxed{\mathbf{h} = g(W^T\mathbf{x} + \mathbf{b})}$$

where W contains the layer's weights, $\mathbf{b}$ contains its biases, g is the activation function, and $\mathbf{h}$ contains the resulting hidden activations.

7.1 Calculate a hidden layer using ReLU

Let the input be $\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. For an illustrative hidden layer, define:

$$z_1 = x_1 + x_2 - 1 = 2$$

$$z_2 = x_1 - x_2 = 1$$

These are linear scores. The activation function transforms each score into a hidden activation. We will use the Rectified Linear Unit (ReLU).

8 Section 6.3.1 — ReLU and generalization

8.1 What is ReLU?

The ReLU activation function is defined as

$$\boxed{g(z) = \max(0, z)}$$

It follows a simple rule:

$$g(z) = \begin{cases} 0, & z \leq 0 \\ z, & z > 0 \end{cases}$$

Apply ReLU to our two hidden-unit scores: $h_1 = \text{ReLU}(2) = 2$ and $h_2 = \text{ReLU}(1) = 1$. Therefore, the hidden representation is

$$\boxed{\mathbf{h} = [2, 1]^T}$$

8.2 ReLU and sigmoid comparison

- **ReLU:** Returns zero for negative input, passes positive input unchanged, has derivative 1 when $z > 0$ and 0 when $z < 0$, not differentiable at zero (handled by subgradient convention).
- **Sigmoid:** Produces output between 0 and 1, useful for binary probability, saturates toward 0 and 1 for extreme inputs, can produce very small gradients.

Goodfellow identifies ReLU as an excellent default for hidden units because its active region behaves linearly and its gradient does not diminish merely because the input is very positive.

Linear score z	**ReLU output** $\max(0, z)$	**Interpretation**
-2	0	Unit inactive for this input
-0.5	0	Unit inactive for this input
0	0	Output is zero
0.5	0.5	Positive activation
2	2	Positive activation

8.3 ReLU in a neural network: a numerical example

8.4 Why does ReLU help training?

Consider the derivative of ReLU:

$$g'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

When a ReLU unit is active, its derivative is 1. This allows the gradient to pass through that activation without the saturation problem associated with sigmoid hidden units. ReLU is also piecewise linear.

8.5 What does generalization mean?

Generalization is the ability of a trained model to perform well on previously unseen examples drawn from the relevant data distribution. ReLU does not guarantee generalization; it helps make hidden layers easier to optimize, but generalization also depends on training data, model capacity, architecture, and validation.

9 Put all the concepts together

Concept	**Purpose**
Gradient-based learning	Optimizes the neural network's parameters
Cross-entropy	Measures mismatch between predicted output distribution and target
Logistic regression	Produces binary-class probability from a linear score
Sigmoid output	Represents a Bernoulli probability
Softmax output	Represents a probability distribution across mutually exclusive classes
Hidden units	Learn intermediate feature representations
ReLU	Introduces nonlinearity while allowing gradients to pass
Generalization	Measures performance on unseen data

Primary reading: Goodfellow, Bengio and Courville — Deep Learning, Chapter 6: Deep Feedforward Networks. Focus on Sections 6.2, 6.2.1, 6.2.2.2, 6.2.2.3, 6.3 and 6.3.1.